Evals — Measuring Whether a Change Actually Helped

Build Week

Build automatic tests that score every change, so you never have to guess whether your app improved.

At a glance

Module	Week 6 · Module 3 — Evals & Error Analysis (The Core)
Format	Build week — hands-on, one deliverable
Deliverable	A one-command test set, a validated judge, and before/after scores per problem type
Mentor review	Friday — informal, no marks
Evaluated	Week 7 · Monday

What this week is about

You can name the problems now. This week you build the machine that catches them automatically — a set of test cases that give every future change a score instead of a gut feeling. This is the single most important skill in the whole program.

Why it matters

Why do I need automatic tests for an AI app? Every time you change a prompt, something might get better and something else worse. Without tests, you're guessing. With them, you get a number.	**What does "LLM-as-judge" mean?** Using one AI call to grade another app's answer — useful for things a simple rule can't check, like tone or helpfulness.	**Why check the judge against a human first?** An AI judge you never checked is just a confident number nobody trusts. You confirm it agrees with your own grading before relying on it.

What you'll learn

- Building a set of test cases: real questions with a way to score the answer
- Turning last week's real failures into permanent tests, so they can't come back unnoticed
- Simple checks a rule can do (is a source present? did it refuse when it should?) — do these first, they're free
- Using an AI to judge the harder stuff (tone, helpfulness) when a rule can't
- Checking the AI judge agrees with your own grading before you trust it
- Measuring a change with a before-and-after score, per problem type

Topics covered

The exact concepts to study.

• Eval sets	• Regression tests from failures
• Assertion checks	• LLM-as-judge
• G-Eval	• Binary vs 1–10 scoring
• Judge validation (human agreement)	• RAGAS: faithfulness
• RAGAS: answer relevancy	• RAGAS: context precision & recall
• Before/after deltas	

Your task this week

Evaluated: Week 7 · Monday

Build a small test set that runs with one command and scores your app's answers. Make one improvement and show the score went up — with a before-and-after number, per problem type.

Assigned tracks

Track	Topic	Deliverable
A	Customer support tickets	Validate the ticket-reply judge before you trust its number
B	Recipes & food	Validate the substitution judge before you trust its number
C	HR policy	Validate the policy-answer judge before you trust its number
D	Insurance claims	Validate the claim-summary judge before you trust its number
E	Developer documentation	Validate the docs-answer judge before you trust its number
F	Legal contracts	Validate the clause-answer judge before you trust its number

Same task, six different topics — so everyone works on their own version. Download whichever you are assigned.

How your mentor checks

Friday · no marks

A quick review of your task. Not a test — just a few things your mentor looks at, so you can fix anything before Monday.

- Does the test set run with a single command?
- Are last week's real failures included as tests?
- If they used an AI judge, did they check it agrees with their own grading first?
- Is there a before-and-after score proving the change helped?